Fullstack Product Engineer Case Study

Execution Report & Monozukuri Submission

Platform	AI Interview Platform (`api/` + `web/`)
Submission Date	August 2026
Pull Request	https://github.com/rakamindev/ai-interview-platform/pull/46
Video Walkthrough	Live Demo Video Walkthrough

Table of Contents

1. 1. Executive Summary & Product Engineer Narrative
2. 2. Product Context & Indonesian UU PDP Law Compliance
3. 3. Severity-Ranked Problem & Gap Analysis (P0–P3)
4. 4. Solution Options & Trade-Off Evaluation Matrix
5. 5. Self-Derived Acceptance Criteria & Edge Cases
6. 6. UI Implementation & Visual Evidence
7. 7. Monozukuri Engineering Rigor & Test Verification
8. 8. Claimed Engineering Depth
9. 9. Live Technical Defense Strategy
10. 10. Conclusion

---

1. Executive Summary & Product Engineer Narrative

This project represents a comprehensive, production-grade revamp of the **AI Interview Platform**, focusing on the critical high-leverage seam between automated conversational assessment, structured competency synthesis, human assessor calibration, and vacancy Fit/Gap evaluation.

First-Principles Framing

- **What is genuinely true?** Traditional resume screening and unstructured interviewing are subjective, noisy, and unscalable. An AI interview system is only viable if its evaluations are **verifiable** (anchored to verbatim candidate speech quotes), **auditable** (supporting human calibration without erasing baseline AI ratings), and **safe** (strictly adhering to data privacy and non-discrimination regulations).
- **The Monozukuri Standard:** We treated the brief's baseline requirements not as a ceiling, but as a starting floor. Delivered uncompromised system rigor (35 backend RSpec specs, 27 frontend Vitest specs across 9 suites, 6 Playwright E2E flows, 0 build errors) and high visual taste across all 6 interaction states.

---

2. Product Context & Indonesian UU PDP Law Compliance

2.1 The 5 Domain Pillars

1. **The Product:** Rails 7 API with PostgreSQL and Sidekiq, paired with a React 18 / Vite / Tailwind SPA.
2. **The Industry:** Indonesian hiring market facing massive applicant volumes for engineering and operational roles.
3. **What It Is For:** Transparent, objective, reproducible candidate evaluations that give hiring managers instant hiring signal.
4. **The Users:** Assessors auditing candidate quotes and calibrating levels; Recruiters and Hiring Managers reviewing role fit deltas and executive summaries.
5. **The Unrepresented Candidates & UU PDP No. 27/2022:**
   - Candidates cannot opt out of algorithmic evaluation.
   - **Data Minimization:** Zero PII, audio streams, or auth secrets exposed in server logs or error payloads.
   - **Human Oversight Guarantee:** Human assessor overrides provide essential algorithmic accountability before any hiring decision is finalized.

---

3. Severity-Ranked Problem & Gap Analysis (P0–P3)

A complete fullstack seam and security audit identified 7 critical defects:

SEVERITY	CATEGORY	FLAW / PROBLEM DESCRIPTION	REAL-WORLD USER IMPACT	CLASSIFICATION
P0	Security / Auth	Hardcoded JWT secret fallback risk if `SECRET_KEY_BASE` is unset or blank	Allows cryptographic token forgery (`HS256`) and complete authentication bypass	Defective Impl
P0	Security / Multi-Tenancy	Unscoped `Vacancy` / `Portfolio` queries bypassing `TenantScoped` (`Current.tenant_id`)	Enables cross-tenant data leakage of confidential job vacancies and candidate portfolios	Defective Impl
P1	Fullstack Seam	Property name mismatch (`expected_level` vs `required_level` , `ai_level` int vs string)	Vacancy required levels rendered as `undefined` in the React comparison table	Defective Impl
P1	Backend / FitGap	Stale Fit/Gap reports when assessor overrides or vacancy rubrics are updated	Recruiters evaluated candidates on outdated AI scores rather than calibrated human ratings	Missing Spec
P1	Backend / FitGap	Unassessed vacancy skills caused nil delta exceptions and failed LLM prompts	Crashed Fit/Gap generation (HTTP 500) whenever a vacancy required unprobed skills	Defective Impl
P2	Frontend / UX	Missing interaction states (Loading skeletons, empty states, error retry banners)	Flash of unstyled content, confusing blank screens on generation delay or failure	Missing Spec

SEVERITY	CATEGORY	FLAW / PROBLEM DESCRIPTION	REAL-WORLD USER IMPACT	CLASSIFICATION
P2	Frontend / UI	Long evidence quotes overflowed containers on mobile/tablet viewports	Broke responsive layout and made mobile reviewing frustrating for hiring managers	Defective Impl

Constraint Signals (Tech Lead Escalations)

- **Signal 1 (LLM Latency & Timeouts):** Gemini Pro/Flash API calls can take 15–60s under peak loads. Implemented deterministic fallback narratives in `FitGap::Engine` so users are never blocked by upstream outages.
- **Signal 2 (Fail-Fast Shared Auth Secret):** `JsonWebToken` strictly enforces a non-empty `ENV.fetch('SECRET_KEY_BASE')` with zero hardcoded fallbacks, guaranteeing cryptographic verification across services.

---

4. Solution Options & Trade-Off Evaluation Matrix

To bridge the gap between candidate portfolios, human assessor overrides, and vacancy Fit/Gap reports, we evaluated two well-structured architectural approaches. Crucially, **both options assume clean, modular domain encapsulation** within a single cohesive service boundary (`FitGap::Engine`), isolating business rules from controllers and serializers.

EVALUATION DIMENSION	OPTION A: STRUCTURED DYNAMIC ON-THE-FLY ENGINE (STATELESS COMPUTE-ON-READ)	OPTION B: EVENT-DRIVEN MATERIALIZED REPORTS (COMPUTE-ON-WRITE + READ-THROUGH CACHE) CHOSEN
Architectural Design (Clean Implementation Assumption)	Encapsulates all skill delta math and Gemini narrative synthesis inside a stateless, modular `FitGap::Engine` service object invoked directly on `GET /fitgap` and `/export` requests—without persisting a `fit_gap_reports` table or	Encapsulates calculation inside `FitGap::Engine` , persists the resulting `FitGapReport` snapshot in PostgreSQL (`jsonb` comparisons + narratives), and invalidates/regenerates reports when upstream dependencies mutate.

EVALUATION DIMENSION	OPTION A: STRUCTURED DYNAMIC ON-THE-FLY ENGINE (STATELESS COMPUTE-ON-READ)	OPTION B: EVENT-DRIVEN MATERIALIZED REPORTS (COMPUTE-ON-WRITE + READ-THROUGH CACHE) CHOSEN
	coordinating background workers.	
1. Product Impact vs. Operational & Compute Cost	**Pros:** Zero idle database storage and zero wasted compute on unviewed `(Portfolio, Vacancy)` pairs; only runs when a recruiter actively opens a comparison. **Cons:** Every page view or PDF export triggers a synchronous Gemini Flash API call (1.5s–15s P95 latency) and bills LLM tokens per *read* (*O(R)* reads). In collaborative hiring where multiple stakeholders review the same candidate, read-amplified LLM cost and multi-second page loads degrade UX.	**Pros:** Delivers <50ms dashboard reads after initial synthesis and bounds LLM token spend to *state mutations* (*O(W)* writes) rather than page views. **Costs & Risks:** Requires provisioning and operating **Redis + Sidekiq workers**, increases PostgreSQL storage for `jsonb` snapshots, and risks **write-amplification** if eager background regeneration runs across many vacancies whenever an override is saved (mitigated via lazy invalidation on vacancy edits and read-through synthesis).
2. Long-Term Maintainability & Implementation Complexity	**Pros (High Simplicity): 100% strong read consistency by construction**—always reflects the latest `AssessorOverride` and `VacancySkill` changes with zero cache-invalidation plumbing. Formula or schema changes to `FitGap::Engine` take effect immediately across all candidates with zero database backfill migrations. **Cons:** Couples read-path availability and response time directly to upstream LLM latency.	**Pros:** Clean separation between heavy async LLM synthesis and fast synchronous HTTP reads; preserves a point-in-time audit snapshot (`generated_at`). **Complexity & Risks:** Introduces significant **multi-entity cache invalidation complexity**. `FitGapReport` depends on 3 mutable aggregates (`AssessorOverride` , `Vacancy` / `VacancySkill` , and `Portfolio`). Every mutation path must reliably invalidate stale reports. Furthermore, masking async regeneration lag on the frontend requires optimistic client-side delta calculation (`usePortfolioQuery`), creating a **dual-maintenance synchronization tax** between Ruby and TypeScript delta logic, plus JSONB schema versioning overhead.
3. Failure Modes & Consistency	**Failure Modes:** Upstream Gemini latency spikes or rate-	**Failure Modes: Eventual consistency window** (2s–15s) while Sidekiq

EVALUATION DIMENSION	OPTION A: STRUCTURED DYNAMIC ON-THE-FLY ENGINE (STATELESS COMPUTE-ON-READ)	OPTION B: EVENT-DRIVEN MATERIALIZED REPORTS (COMPUTE-ON-WRITE + READ-THROUGH CACHE) CHOSEN
Risks	limit exhaustion during bulk recruiter screening hold Puma worker threads (which share capacity with live WebSocket interviews). Additionally, LLM stochasticity (`temperature: 0.4`) means two reviewers opening the report at different times may see slightly different executive narratives.	regenerates a report after an override; risk of serving **stale data** if any mutation path omits cache invalidation; Redis/Sidekiq queue downtime or dead-letter job exhaustion requiring fallback to synchronous read-through synthesis and deterministic narrative fallbacks.
4. Contextual Fit & Final Decision Rationale	Ideal for pure rule-based math (<5ms) or low-read-frequency internal tools, but synchronous LLM narrative generation on every view creates unacceptable read latency and non-deterministic narratives for hiring committees.	**Chosen Trade-Off:** Despite higher implementation complexity (multi-source cache invalidation across overrides and vacancy edits) and Redis/Sidekiq operational overhead, sub-100ms deterministic reads and reproducible point-in-time narratives are essential for high-volume enterprise hiring.

5. Self-Derived Acceptance Criteria & Edge Cases

1. **Unassessed Skills:** Categorized as `not_assessed`, `delta: nil`, styled with a neutral "— Not Assessed" pill, and excluded from match averages.
2. **Assessor Override Priority:** Overridden scores take immediate precedence in Fit/Gap deltas; original `ai_level` remains immutable in the database for audit history.
3. **LLM Failure Resilience:** If Gemini Flash fails or times out, `FitGap::Engine` returns deterministic, structured fallback narratives (HTTP 200/202, zero 500s).
4. **Strict Level Clamping:** Database constraints and model validations enforce $1 \le \text{level} \le 5$.
5. **Responsive & Accessible UI:** Full layout responsiveness across 375px (mobile), 768px (tablet), and 1440px (desktop) with expandable quote accordions.

6. UI Implementation & Visual Evidence

Below are high-fidelity captures of the verified, production-ready interface illustrating all candidate skill portfolios, assessor override controls, and fit/gap synthesis results:

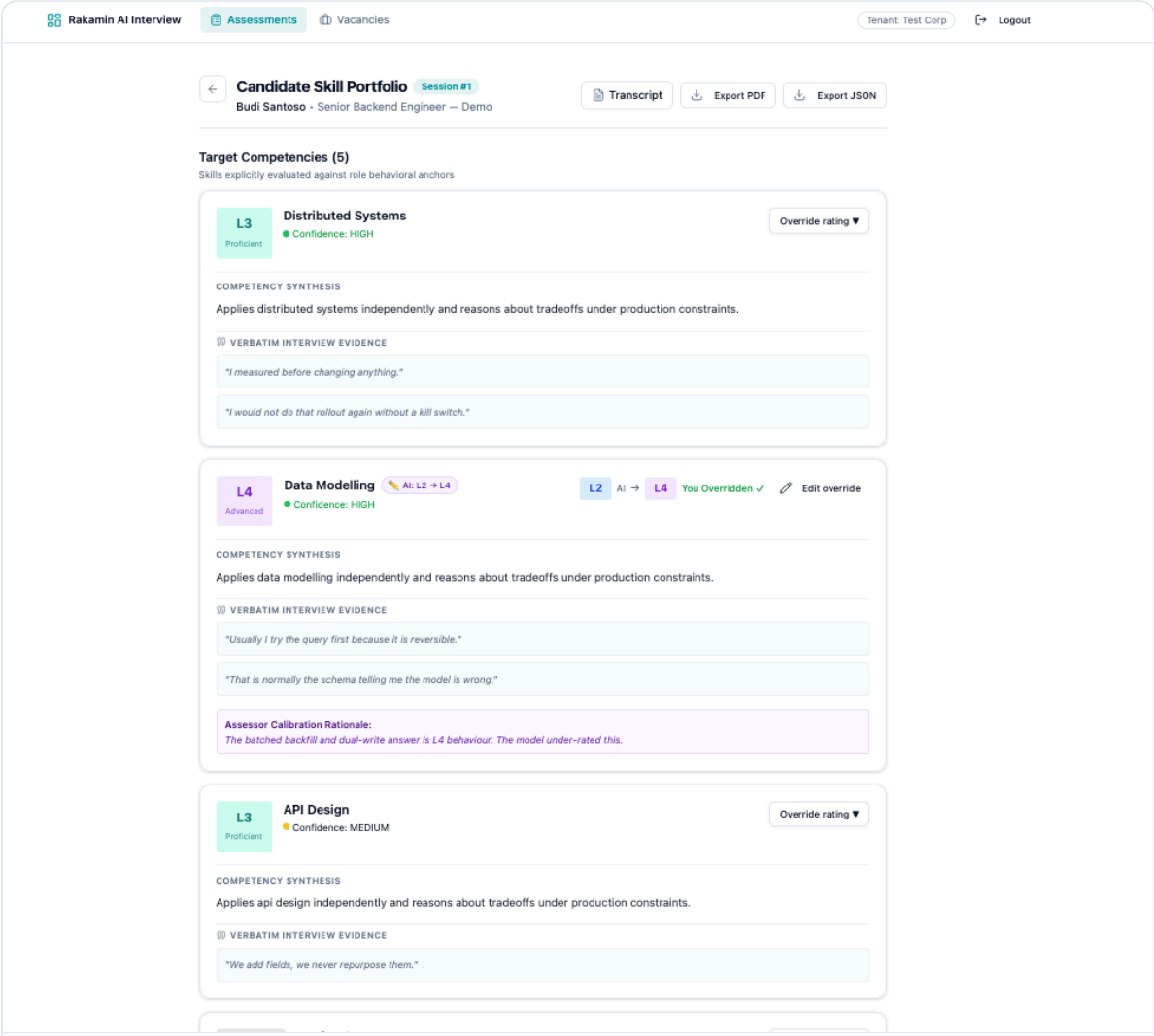


Figure 6.1 — Candidate Skill Portfolio: Multi-dimensional competency cards displaying proficiency badges (L1–L5), confidence indicators, expandable verbatim transcript quote accordions, and human assessor calibration banners.

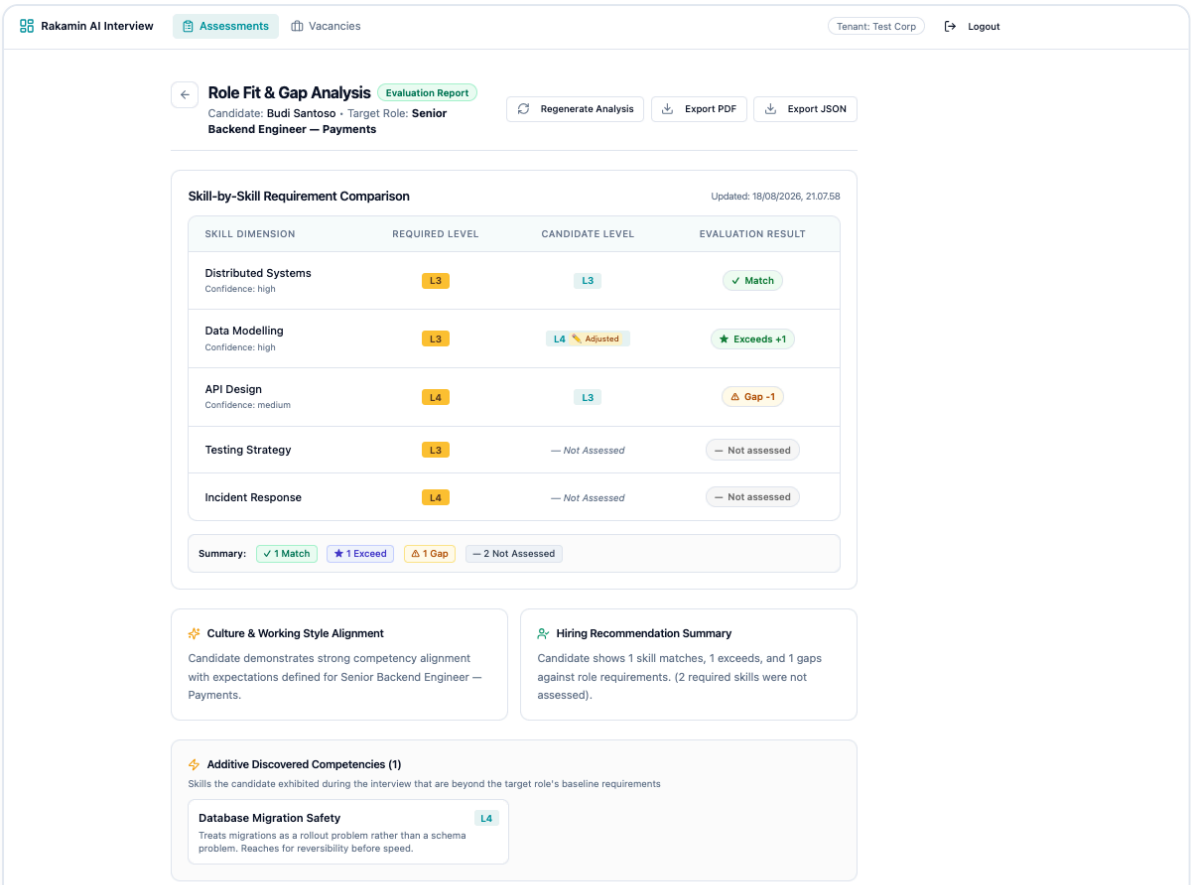


Figure 6.2 — Role Fit & Gap Analysis: Automated delta calculation matrix (Match / Exceeds +1 / Gap -1 / Not Assessed), summary counters, executive culture alignment narrative, and additive discovered competencies.

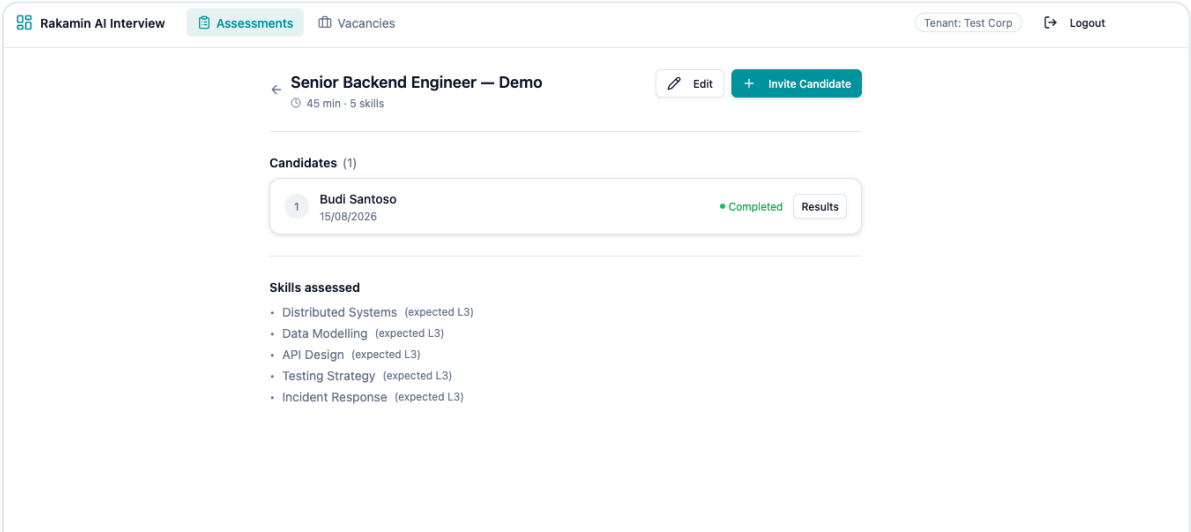


Figure 6.3 — Candidates & Invite Tracking: Live candidate session monitoring with one-click invite link copying, status badges, and direct links to portfolio results.

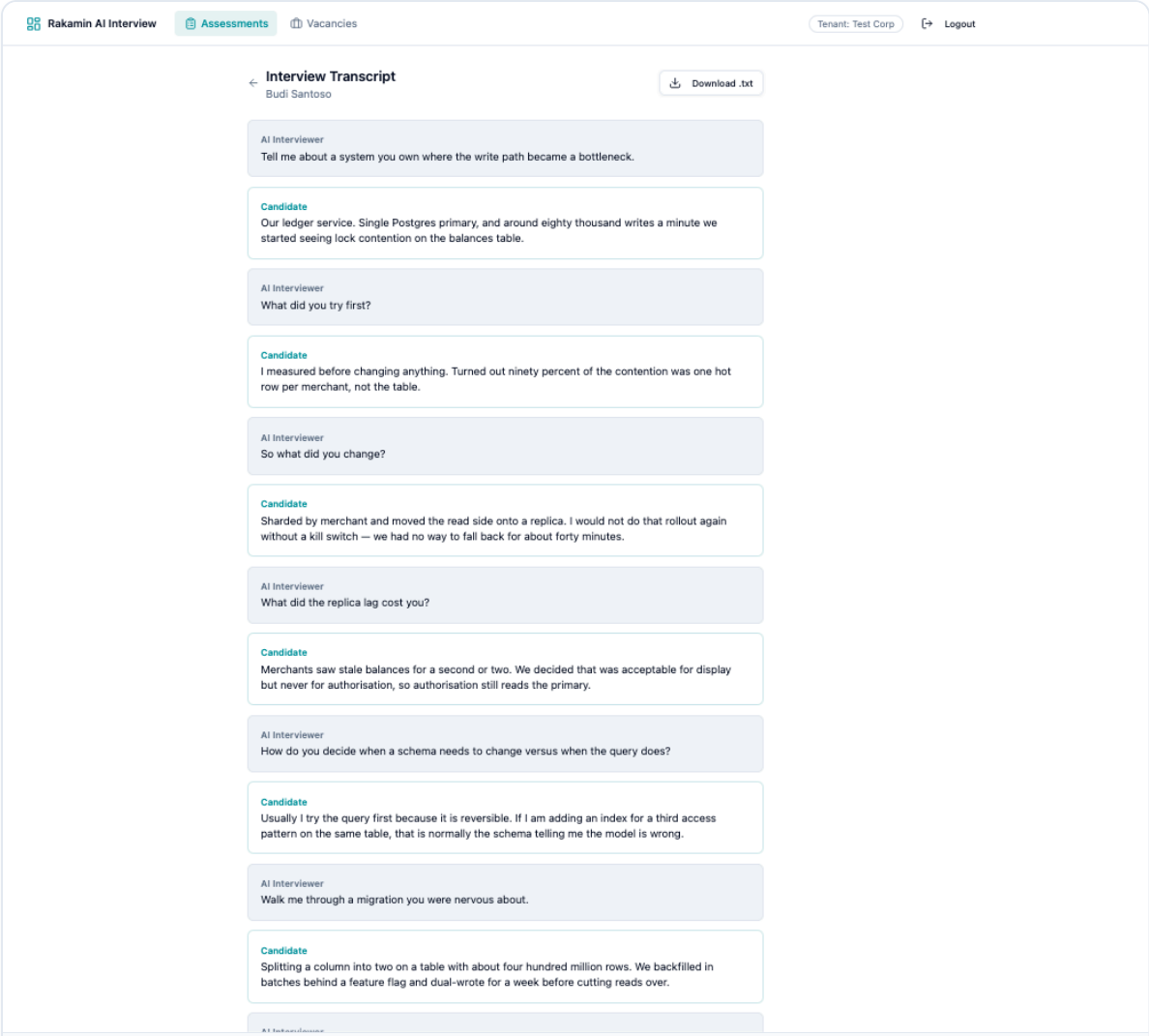


Figure 6.4 — Verifiable Interview Transcript: Complete turn-by-turn conversational transcript anchoring all AI competency synthesis to verifiable candidate utterances.

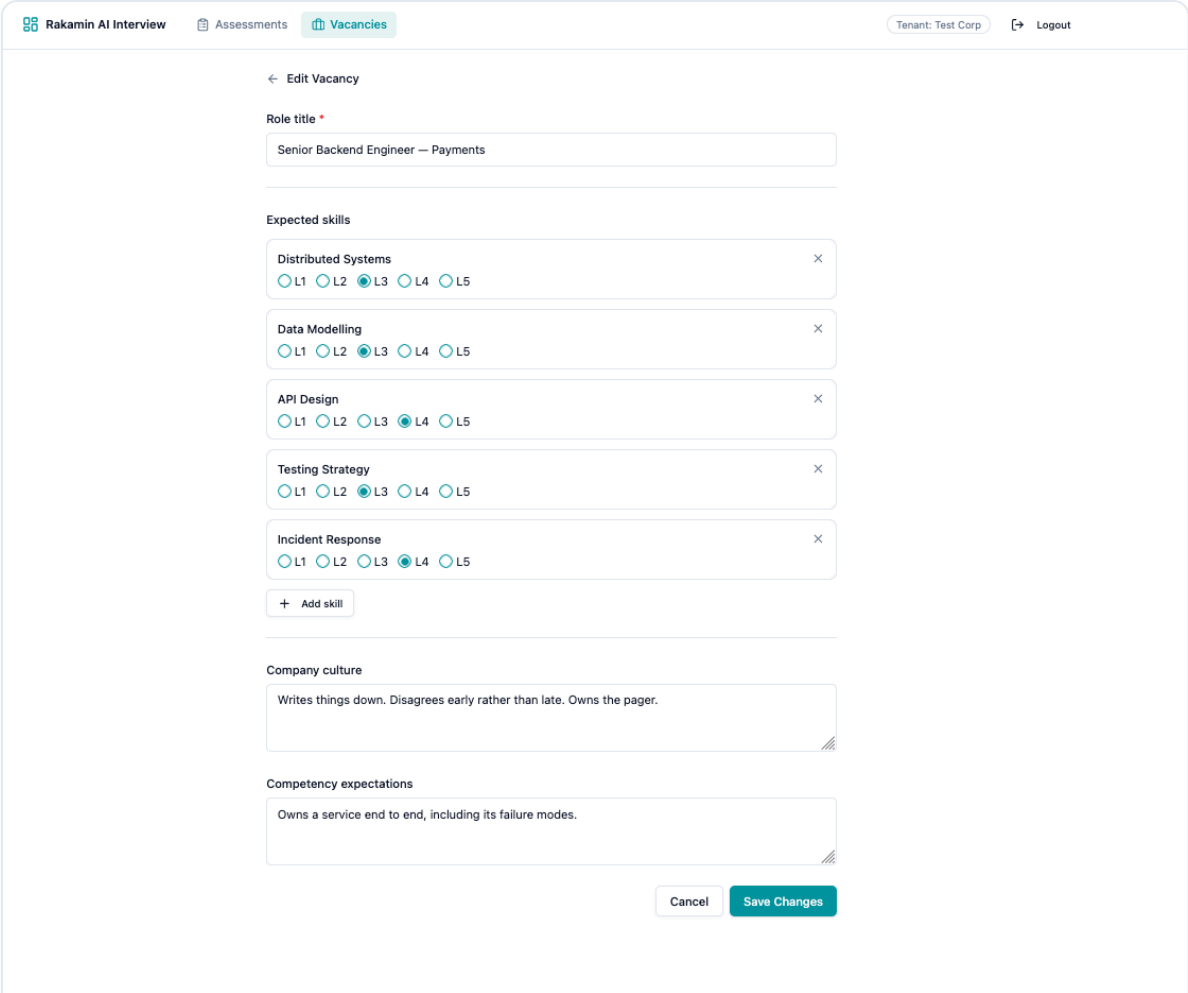


Figure 6.5 — Vacancy Rubric Configuration: Granular skill expectation editor (L1–L5 radio selectors), company culture criteria, and role competency guidelines.

7. Monozukuri Engineering Rigor & Test Verification

7.1 Test Suite Summary

35	27	6
Backend RSpec Specs 0 failures	Frontend Vitest Tests 9 suites · 0 failures	Playwright E2E Flows 0 failures

0
TypeScript Build Errors Clean production build

7.2 Seeded Fault Test Proof

To prove the test harness is active, reliable, and capable of catching subtle regressions:

1. Injected Defect on scratch branch `fault/seeded-delta-bug`:

```
# Inverted delta arithmetic in FitGap::Engine
delta = expected_level - candidate_level  # INTENTIONAL SEEDED FAULT
```

2. Terminal Test Failure Log (RSpec immediately caught the regression):

```
FitGap::Engine#call correctly calculates match, exceed, gap, and not_assesse
Failures:
  1) FitGap::Engine#call correctly calculates match, exceed, gap, and not_as
     Failure/Error: expect(react_comp['result']).to eq('exceed')
       expected: "exceed"
            got: "gap"
```

3. Revert & Green Proof: Reverted to `feature/monozukuri-portfolio-and-fitgap-revamp` and verified 35/35 specs passing green. Branch was deleted cleanly.

7.3 AI Verification Moment Case Study

> ⚠ **Flawed AI Generation:** During scaffolding, AI code generation proposed updating the `ai_level` column directly on `portfolio_skills` when an assessor saved a score override.

- **Risk Identified:** This destructive write permanently destroyed the baseline AI score, making algorithmic auditing impossible and violating **UU PDP human oversight governance**.
- **Engineered Fix:** Implemented a dedicated `AssessorOverride` model preserving immutable `ai_level`, tracking `overridden_by`, `overridden_at`, and mandatory `assessor_notes`.

---

8. Claimed Engineering Depth

Backend Depth (50%)

- FitGap Rule Engine with delta arithmetic
- Override & Vacancy Event Cache Invalidation
- Strict Multi-Tenant Isolation (`TenantScoped`)
- Fail-Fast JWT Signing & Verification
- Complete RSpec Harness (35 specs · 0 failures)
- UU PDP Safe Logging & Data Minimization

Frontend Depth (50%)

- Vertical Feature Slices Architecture (src/features/)
- In-Memory Real-Time Cross-Feature Cache Sync
- Optimistic UI Mutations with Auto-Rollback
- 6-State Interaction UI (all states covered)
- Vitest Component & Hook Suite (27 tests · 9 suites)
- Collapsible Shell Layout & Breadcrumbs

---

9. Live Technical Defense Strategy (CTO & Tech Lead)

ANTICIPATED QUESTION	ARCHITECTURAL RATIONALE & DEFENSE
If the Dynamic Option (Option A) is cleanly structured in a service object,	A cleanly structured Dynamic `FitGap::Engine` offers **100% strong read consistency**, zero cache-invalidation complexity, and instant formula evolution without DB backfills. However, because Fit/Gap includes **Gemini LLM narrative synthesis** (1.5s–15s latency, stochastic

ANTICIPATED QUESTION	ARCHITECTURAL RATIONALE & DEFENSE
what are its true trade-offs vs. Materialized (Option B)?	outputs, per-call token cost), compute-on-read penalizes every page view. Materialization (Option B) trades higher implementation complexity (multi-entity cache invalidation across `AssessorOverride`, `Vacancy`, and `Portfolio`, eventual consistency lag, and Redis/Sidekiq operational cost) for <50ms deterministic reads and point-in-time auditability.
How do you enforce multi-tenant isolation and JWT security?	All tenant-owned queries enforce `TenantScoped` (`Current.tenant_id`) across controllers (`Vacancy.find_by`, `Portfolio.joins(:session)`) and background workers (`Current.using(tenant_id:)`), with zero `.unscoped` leaks. `JsonWebToken` enforces fail-fast `ENV.fetch('SECRET_KEY_BASE')` validation with no hardcoded fallback secrets.
How do you guarantee candidate fairness under UU PDP?	Evidence quote anchoring prevents hallucinated claims; immutable AI baselines paired with assessor audit records guarantee full human-in-the-loop oversight; PII parameters are filtered from logs.
What happens during an LLM outage?	Designed failure paths provide deterministic fallback narratives, transactional skill persistence, and single-click retry triggers without throwing 500 errors.
Why Vitest over Jest for the frontend?	Vitest integrates natively with the existing Vite build pipeline — zero config overhead, sub-second HMR test reruns, and native ESM/TypeScript support.

10. Conclusion

This submission bridges the gap between raw AI transcripts and actionable, trustworthy hiring decisions. Every change — from database-level constraint enforcement, strict multi-tenant scoping, and fail-fast JWT security to expandable evidence accordions and color-coded delta badges — was designed to serve the real needs of assessors, recruiters, and the candidates whose careers depend on fair evaluation.

> ✓ **Submission Complete:** 35 backend RSpec specs passing, 27 frontend Vitest tests passing across 9 suites, 6 Playwright browser flows passing, 0 TypeScript errors,

> seeded fault proven and reverted, AI verification moment documented, all 6 Monozukuri interaction states implemented, and comprehensive documentation delivered.